8-Day GenAI Placement Bootcamp Plan

To get placement-ready for on-campus software interviews without running anything locally or getting bogged down in heavy Machine Learning math, treat this course as Applied AI Engineering (APIs + Backend Logic) rather than Data Science. In on-campus interviews, recruiters rarely ask you to derive neural network equations for GenAI roles; instead, they test if you understand **how to orchestrate LLM APIs, structure JSON outputs, prevent hallucinations using RAG, and build tool-calling agents**.

1. Your "Zero-Local" Cloud Setup

To avoid installing Python environments, dependencies, or heavy models on your laptop, use this 100% browser-based stack:

- **Cloud IDE (Primary): GitHub Codespaces** (Free for 60 hours/month, or free via GitHub Student Developer Pack). Create a single GitHub repository called `genai-placement-bootcamp` and open it in a Codespace. It gives you a full cloud VS Code terminal, lets you preview Streamlit UIs via browser port-forwarding, and lets you push directly to GitHub in seconds.
- **Scratchpad (Optional): Google Colab** for quick single-file testing of document loaders and text splitters.
- **Free Cloud LLM APIs (Instead of paid OpenAI or local models):**
    - **Google Gemini API (`langchain-google-genai`):** Free tier gives you both Chat Models (`gemini-2.5-flash`) and Embedding Models (`text-embedding-004`).
    - **Groq Cloud API (`langchain-groq`):** Free, ultra-fast cloud inference for open-source models like Llama-3.

2. How to Tackle the ML Parts (Zero-Math Strategy)

You only encounter **four core ML concepts** across these ~11.5 hours of video. Do not take a separate ML course—use these targeted 10-minute conceptual resources when those timestamps come up:

ML Concept in Course	Where it Appears	What to Read / Watch (10–15 mins)	What to Say in an On-Campus Interview
1. Tokens, Temperature & Context Window	Video 1 (`00:11:08`)	Read OpenAI's *"What are tokens and how to count them?"* article or watch a 5-min video on *LLM Temperature & Top-P*.	*"Tokens are sub-word chunks (~0.75 words). Temperature controls output randomness: I set it to `0.0` for deterministic JSON/code generation and `0.7+` for creative chatbots."*
2. Text Embeddings	Video 1 (`01:52:16`) & Video 2 (`01:42:29`)	Watch **StatQuest: "Word Embedding and Word2Vec, Clearly Explained"** (first 10 mins on YouTube).	*"An embedding model converts text into a fixed-size array of floating-point numbers (a vector) so that texts with similar meanings map close together in multi-dimensional space."*
3. Cosine Similarity & Vector Search	Video 2 (`01:42:29`)	Read Pinecone's short blog: *"What is a Vector Database & Similarity Search?"*	*"Keyword search matches exact strings, whereas Vector Search uses Cosine Similarity (measuring the angle between two vectors) to retrieve semantically similar chunks even if exact words don't match."*
4. RAG vs. Fine-Tuning	Video 2 (`00:01:48`)	Watch IBM Technology's 8-min video: *"RAG vs. Fine-Tuning"*.	*"Fine-tuning updates model weights to teach a new style or format, which is expensive and still hallucinates facts. RAG injects live external documents into the prompt at runtime, making it cheaper, citable, and easy to update."*

3. Master Triage: Watch vs. Implement

Use this lookup table so you don't waste hours implementing low-ROI setup steps or local model downloads:

Module	Watch Only (1.25x–1.5x Speed for Theory)	Must Implement (Hands-On in Cloud)
Part 1: Chatbots & Prompts	* Intro & Roadmap (`00:00:00 - 00:35:27`) * Local Venv (`00:35:27 - 00:41:02` — *Codespaces handles this*) * Hugging Face & Running Models Locally (`01:28:24 - 01:52:16` — *Watch for awareness, never download locally*)	* Folder Structure, API Keys & `requirements.txt` (`00:41:02 - 01:07:46`) * Chat Models & Embeddings in Python (`01:07:46 - 01:28:24`, `01:52:16 - 02:08:56`) * Chatbot with Memory & UI (`02:08:56 - 02:55:32`) * Prompt Templates & Structured Output (`02:55:32 - 03:50:54`)
Part 2: RAG Pipeline	* Intro, Plan & Setup (`00:00:00 - 00:30:10`) * Obscure Document Loaders & exotic Text Splitters (watch the theory of all, only code the main ones)	* `PyPDFLoader` & `WebBaseLoader` (`00:30:10 - 00:58:45`) * `RecursiveCharacterTextSplitter` (`00:58:45 - 01:42:29`) * Chroma/FAISS In-Memory Vector Store (`01:42:29 - 02:31:37`) * Similarity/MMR Retriever & Full RAG (`02:31:37 - 03:41:24`)
Part 3: Runnables & Agents	* Series Intro (`0:00 - 4:15`) * Edge-case Runnable primitives (watch how they work conceptually)	* Core LCEL Pipe (`|`) & Code Generator Chain (`4:17 - 1:26:40`) * Tavily Search & Custom `@tool` functions (`1:26:42 - 1:58:20`) * Tool Binding & Manual Agent Loop (`1:58:24 - 2:43:00`) * OpenWeather Agent & `create_agent` (`2:43:02 - 4:06:10`)

4. The 8-Day Placement Bootcamp Schedule

Day 1: LLM Foundations, Cloud Workspace & First API Call

- **Video Segment (Part 1):** `00:00:00 - 01:28:24` (**Total Watch Time:** 1 hr 28 mins)
- **Execution Plan:**
    a. **Watch Only (1.5x):** `00:00:00 - 00:35:27` (Intro, Prerequisites, Roadmap, and Foundation of LLMs & Chat Models). Note down the difference between standard completion models and chat-tuned models.
    b. **ML Quick Read (10 mins):** Read up on *Tokens, Context Window, and Temperature*.
    c. **Implement in Cloud:** Open GitHub Codespaces (skip local venv at `00:35:27 - 00:41:02`). Set up the folder structure, `.env` file with a free Gemini/Groq API key, and `requirements.txt` (`00:41:02 - 01:07:46`). Write your first Python script invoking the chat model (`01:07:46 - 01:28:24`).
- **Practice Assignment 1:** Write a script `temperature_test.py` that sends the exact same prompt (*"Give me a 1-line pitch for a food delivery app"*) twice—once with `temperature=0.0` and once with `temperature=1.2`. Log the response metadata (input/output token counts) to the console.
- **Git Checkpoint:**git add . && git commit -m "Day 1: Project setup and basic ChatModel invocation" && git push

Day 2: Embeddings, Conversational Memory & Chatbot UI

- **Video Segment (Part 1):** `01:28:24 - 02:55:32` (**Total Watch Time:** 1 hr 27 mins)
- **Execution Plan:**
    a. **Watch Only (1.5x):** `01:28:24 - 01:52:16` (Integrating Hugging Face & Running AI Models Locally). **Do not run models locally.** Just understand why companies sometimes host local quantized models (data privacy/cost at scale) vs. cloud APIs.
    b. **ML Quick Watch (10 mins):** Watch StatQuest's explainer on *Word Embeddings* right before watching `01:52:16 - 02:08:56` (Understanding Embedding Models).
    c. **Implement in Cloud:** Code a quick embedding test (`01:52:16 - 02:08:56`), then build the stateful chatbot using `SystemMessage`, `HumanMessage`, and `AIMessage` history (`02:08:56 - 02:35:51`) and wrap it in the UI (`02:35:51 - 02:55:32`).
- **Practice Assignment 2:** Modify the chatbot's message history array to implement a **"Sliding Window Memory"** that only keeps the `SystemMessage` plus the **last 6 messages** (3 turns) so your token cost never explodes during a long conversation.

- **Git Checkpoint:**git add . && git commit -m "Day 2: Stateful chatbot with sliding-window memory and UI" && git push

Day 3: Prompt Templates, Structured JSON Outputs & RAG Ingestion

- **Video Segment (Part 1):** `02:55:32 - 03:50:54` (55 mins) + **Video Segment (Part 2):** `00:00:00 - 00:58:45` (58 mins) (**Total Watch Time:** 1 hr 53 mins)
- **Execution Plan:**
    a. **Implement in Cloud (High Interview Value):** `02:55:32 - 03:50:54` (Prompt Templates & Structured Output for the CineSage AI Project). In interviews, always highlight **Structured Output (Pydantic schemas)** because real backends need strict JSON from LLMs, not raw markdown paragraphs.
    b. **Watch Only (1.5x):** Part 2 `00:00:00 - 00:30:10` (RAG Intro, Learning Plan, Basic Setup).
    c. **Implement in Cloud:** Part 2 `00:30:10 - 00:58:45` (Document Loaders). Implement `PyPDFLoader` (using a sample PDF like your own resume) and `WebBaseLoader`.
- **Practice Assignment 3:** Build a `resume_parser.py` script that loads a PDF resume using `PyPDFLoader` and uses `with_structured_output()` (Pydantic) to extract `{ "candidate_name": str, "skills": list[str], "projects": list[str], "ats_score_out_of_100": int }`.
- **Git Checkpoint:**git add . && git commit -m "Day 3: CineSage structured output and PDF resume parser" && git push

Day 4: RAG Core — Text Splitters & Vector Stores

- **Video Segment (Part 2):** `00:58:45 - 02:31:37` (**Total Watch Time:** 1 hr 33 mins)
- **Execution Plan:**
    a. **Watch + Selective Implement:** `00:58:45 - 01:42:29` (Text Splitter). Watch the explanation of why naive character splitting breaks sentences, then **only implement `RecursiveCharacterTextSplitter`** with `chunk_size` and `chunk_overlap`.
    b. **ML Quick Read (10 mins):** Read Pinecone's short guide on *Vector Databases & Cosine Similarity*.
    c. **Implement in Cloud:** `01:42:29 - 02:31:37` (Vector Stores). Use `FAISS` (runs in-memory on Colab/Codespaces with `faiss-cpu`) or ephemeral `Chroma` to embed and store your document chunks.

- **Practice Assignment 4:** Chunk your sample PDF using two configs: `(chunk_size=200, chunk_overlap=0)` vs. `(chunk_size=800, chunk_overlap=150)`. Run a similarity search query on both vector stores and document in a `NOTES.md` file why `chunk_overlap` prevents context loss at chunk boundaries.
- **Git Checkpoint:**git add . && git commit -m "Day 4: Text chunking experiments and Vector Store indexing" && git push

Day 5: Retrievers & End-to-End RAG Project Completion

- **Video Segment (Part 2):** `02:31:37 - 03:41:24` (**Total Watch Time:** 1 hr 10 mins)
- **Execution Plan:**
    a. **Watch Theory + Implement Core:** `02:31:37 - 03:11:24` (Retrievers). Pay close attention to **MMR (Maximal Marginal Relevance)**—interviewers love asking *"What if the top 4 chunks returned by your vector database are all near-duplicates?"* (Answer: Use MMR to balance relevance with diversity).
    b. **Implement in Cloud:** `03:11:24 - 03:41:24` (Completing the RAG Project). Connect the Retriever + Prompt Template + Chat Model so the LLM answers strictly from retrieved chunks.
- **Practice Assignment 5:** Add **Hallucination Guardrails & Citations** to your RAG project: update the system prompt so if the answer is not in the retrieved context, the bot replies `"I cannot verify this from the provided documents"`, and appends `[Source Page: X]` using the chunk's `metadata["page"]`.
- **Git Checkpoint:**git add . && git commit -m "Day 5: Complete end-to-end RAG pipeline with source citations" && git push

Day 6: LangChain Expression Language (LCEL / Runnables) & Chains

- **Video Segment (Part 3):** `00:00:00 - 01:26:40` (**Total Watch Time:** 1 hr 26 mins)
- **Execution Plan:**
    a. **Watch Only:** `0:00 - 4:15` (Series Intro & Roadmap).
    b. **Implement Core Primitives:** `4:17 - 59:40` (Runnables: Understanding and using basic components in LangChain). Implement `RunnableSequence` (the `|` pipe operator), `RunnableParallel`, and `RunnablePassthrough`.
    c. **Implement in Cloud:** `59:43 - 1:26:40` (Building a Code Generator chain: Practical implementation).
- **Practice Assignment 6:** Extend the Code Generator chain using `RunnableParallel` so that given a problem description (e.g., *"Binary Search in C++"*), it simultaneously

generates: (1) the clean code implementation, and (2) three edge-case unit tests, returning both in a single dictionary.

- **Git Checkpoint:**git add . && git commit -m "Day 6: LCEL Runnables and parallel Code Generator chain" && git push

Day 7: AI Agents, Custom Tools & Manual Execution Loops

- **Video Segment (Part 3):** `01:26:42 - 02:43:00` (**Total Watch Time:** 1 hr 16 mins)
- **Execution Plan:**
    a. **Implement in Cloud:** `1:26:42 - 1:39:50` (Tools & Tavily Search API setup—get a free Tavily API key) and `1:39:53 - 1:58:20` (Creating Custom Tools with the `@tool` decorator and type-hinted docstrings).
    b. **Implement & Study Deeply:** `1:58:24 - 2:20:05` (Tool Binding and Calling) and `2:20:08 - 2:43:00` (Manual Agent Loop construction). *Crucial Interview Concept:* Notice that an LLM **never executes your Python function directly**—it returns a `tool_calls` JSON payload asking your code to run the function and feed the `ToolMessage` output back to the LLM.
- **Practice Assignment 7:** Create two custom `@tool` functions (`calculate_emi(principal, rate, months)` and `check_loan_eligibility(income, credit_score)`). Build the manual agent loop (`2:20:08 - 2:43:00`) that routes a user's natural language finance query to the right tool and prints each step of the loop.
- **Git Checkpoint:**git add . && git commit -m "Day 7: Custom tools, tool binding, and manual Agent execution loop" && git push

Day 8: Live API Agent Project & Interview Polish

- **Video Segment (Part 3):** `02:43:02 - 04:06:10` (**Total Watch Time:** ~1 hr 04 mins)
- **Execution Plan:**
    a. **Implement in Cloud:** `2:43:02 - 3:07:45` (Integrating OpenWeather API: Practical agent project building).
    b. **Implement & Watch:** `3:27:00 - 4:06:10` (Advanced Agent concepts & `create_agent` method). Refactor your manual loop from Day 7 into the clean `create_agent` workflow.
- **Practice Assignment 8 (Placement Capstone Polish):**
    - Combine **Part 2 (RAG)** and **Part 3 (Agents)** into a single 80-line script `agentic_rag.py`: turn your Day 5 PDF Retriever into a `@tool` alongside the Tavily

Web Search tool and OpenWeather API tool. Now your agent automatically searches your PDF for document-specific questions, checks live weather for city queries, and searches the web for current events.
    - Write a clean `README.md` in your GitHub repo featuring an architecture diagram (ASCII or Mermaid.js) showing **Prompting/Structured Output → RAG Pipeline → Multi-Tool AI Agent**.
- **Git Checkpoint:**git add . && git commit -m "Day 8: Multi-tool Agentic RAG capstone and portfolio README" && git push

5. 5-Minute Interview Cheat Sheet (What You'll Know by Day 8)

If an interviewer asks about this project on your resume, these four answers cover 95% of on-campus GenAI questions:

1. **"How does your RAG pipeline work under the hood?"**
    - *"During ingestion, I load PDFs (`PyPDFLoader`), split them via `RecursiveCharacterTextSplitter` with overlap so context isn't cut mid-sentence, convert chunks into dense vectors using an Embedding API, and index them in a Vector Store. At query time, I embed the user's question, run a similarity/MMR search to fetch the top-K relevant chunks, and pass them into the LLM's system prompt."*
2. **"How do you stop the LLM from returning broken formats to your backend?"**
    - *"Instead of relying on text prompts like 'please return JSON', I bind a Pydantic schema using `.with_structured_output()` so the model's output is strictly validated and typed before my backend processes it."*
3. **"What is the difference between a Chain (LCEL) and an Agent?"**
    - *"In an LCEL Chain (`|`), the sequence of steps is hardcoded by the developer. In an Agent, the LLM acts as a reasoning engine in a loop: it inspects bound `@tool` schemas, decides which tool(s) to call dynamically, observes the `ToolMessage` result, and loops until the task is solved."*
4. **"How does an LLM 'call' an external API like OpenWeather?"**
    - *"The LLM doesn't make HTTP requests itself. When we bind tools to the LLM, we pass the function name, docstring, and JSON parameter schema. The LLM outputs a structured `tool_calls` request containing the arguments (e.g., `city='Pune'`),*

our Python runtime executes the actual OpenWeather API function, and we pass the result back to the LLM to formulate the final answer."